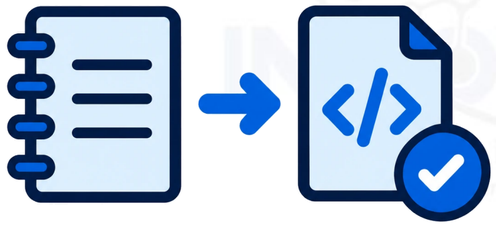
MLOps Specialization Bootcamp

Module 2

Reproducible Projects

Week 2: Git and code quality. Turn a notebook into code a team can trust



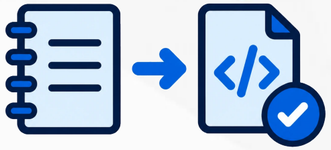


Daniel Islam
Instructor, InnovatiCS MLOps Specialization

INNOVATICS
www.innovatics.ai
Learn Data Science & AI - From Zero to Hero

InnovatiCS 2024. All rights reserved

What you will be able to do



1. Explain why notebooks are hard to reproduce
2. Refactor a notebook into a package with config, a Makefile and tests
3. Branch, commit, push, open a pull request, review, merge and fix a conflict
4. Enforce code quality automatically with ruff, black and pre-commit
5. Test ML code with pytest, including a test that config and code agree

Today

01

Why notebooks break

02

Project layout

03

Git and pull requests

04

Code quality

05

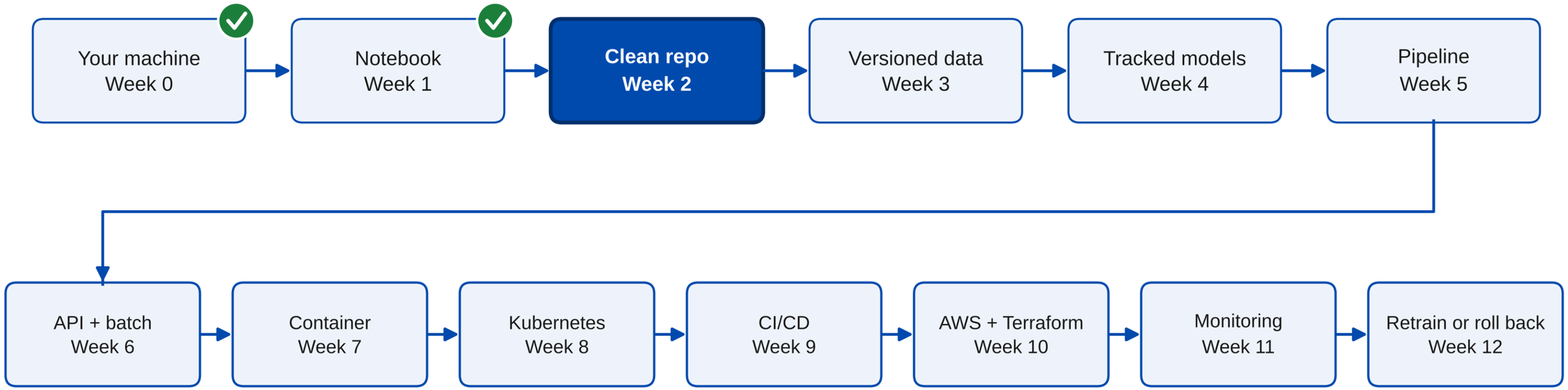
Tests and the check

06

Practical 2

INNOVATICS
www.innovatics.ai
Data Science & AI From Zero to Hero

Where we are



Week 2 turns the notebook into a repository every later week builds on.

01

Why notebooks break

www.innovatics.ai
Learn Data Science & AI - From Zero to Hero

InnovatiCS 2024. All rights reserved

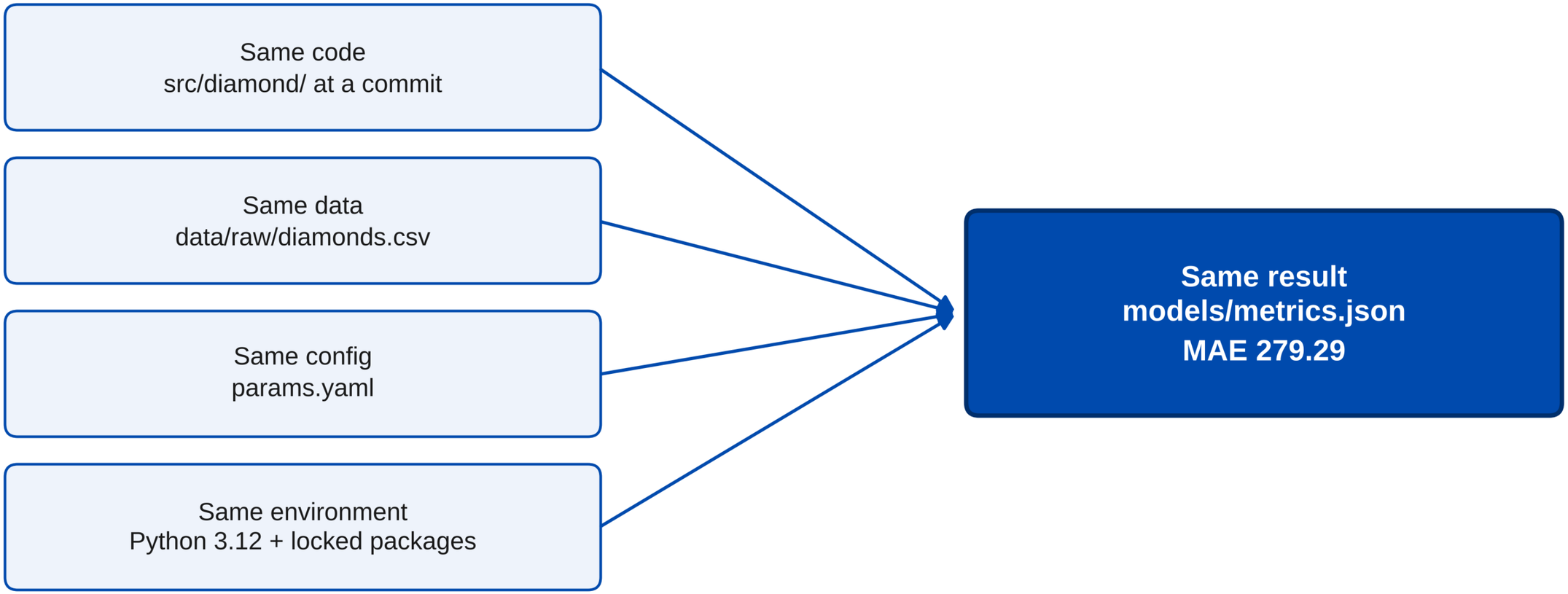
The big question

How do I turn a messy notebook into code a team can **run**, **review** and **trust**?

- Our notebook trains a good model: MAE 279.29 USD on 10,755 holdout rows
- Its own last cell lists what is missing: monolithic, stateful, hardcoded, no tests
- It also cannot be reproduced with one command
- Today we fix all of that without changing the model

www.innovatics.ai
Learn Data Science & AI From Zero to Hero

Reproducible means four things stay the same



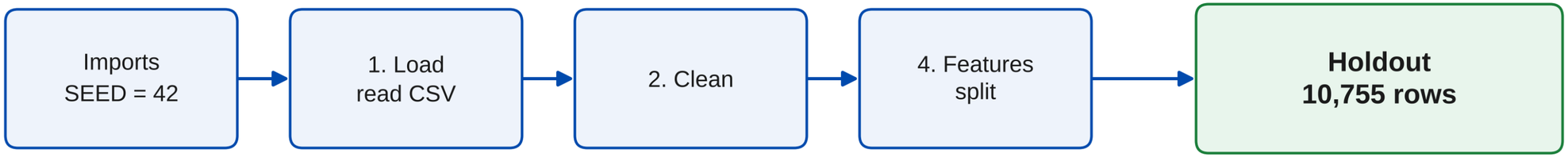
Change any one input and the number can change.

Four notebook problems

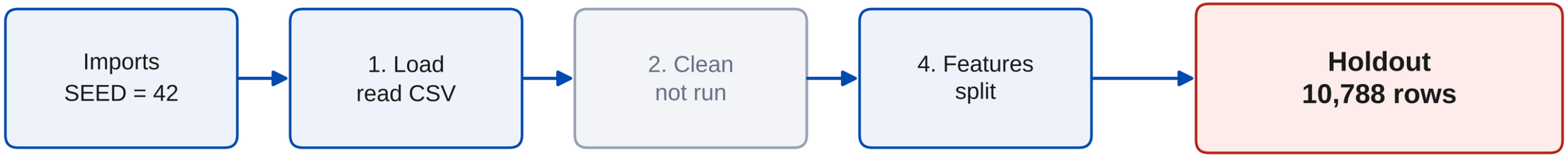
- **Hidden state**: the kernel remembers values no visible cell produces any more
- **Out-of-order runs**: you can run the split before the cleaning, and nothing records it
- **Copy-paste reuse**: the same cleaning code drifts across notebooks
- **No tests**: nothing checks the cleaning, the split or the config
- Plus hardcoded values: `SEED = 42` in the imports cell, `test_size=0.2` in the split

Live: same notebook, different answer

Run all, in order



Restart, skip the cleaning cell



Same notebook. Different run order. Different data in the split.

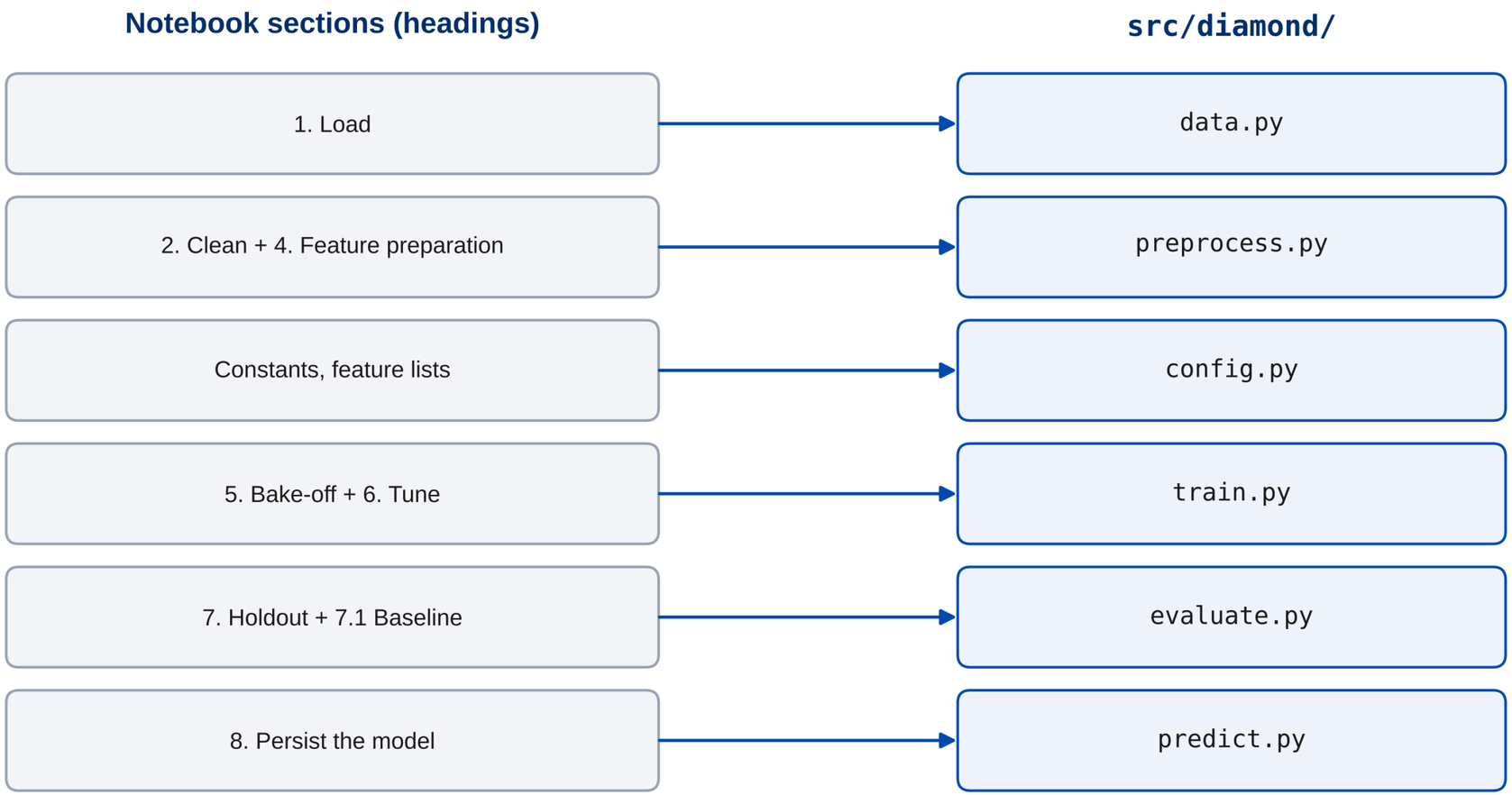
02

Project layout

INNOVATICS
www.innovatics.ai
Learn Data Science & AI - From Zero to Hero

InnovatiCS 2024. All rights reserved

From notebook cells to a package



One module, one job. params.yaml holds the settings.

The project at tag m02

Top level

- `src/diamond/`: the package
- `tests/`: 13 pytest tests
- `params.yaml`: experiment settings
- `pyproject.toml`: pins and tool settings
- `Makefile`: one command per job

Why `src/`

- Tests import the **installed** package
- "Works only from this folder" bugs show up early
- `make install` installs it in editable mode
- Run modules with `python -m diamond.train`

Every setting lives in params.yaml

```yaml
seed: 42
test_size: 0.2

model:
  # Must be a key in diamond.train.candidate_models(). This value is what the
  # code actually trains -- there is no hardcoded model name anywhere in src/.
  name: HistGradientBoosting

  # Winning configuration from the notebook's RandomizedSearchCV
  # (notebooks/01_diamond_exploration.ipynb, section 6). These keys are passed
  # straight through as estimator kwargs, so they must match the estimator's
  # own API -- HistGradientBoosting takes max_iter, not n_estimators.
  # Module 4 wires this same block into MLflow as the logged param set.
  params:
    max_iter: 400
    learning_rate: 0.05
    max_depth: null      # null = unlimited depth (sklearn's None)
```

params.yaml (lines 7-23) at tag m02

`git diff params.yaml` now shows exactly what changed between two experiments.

Missing config must fail loudly

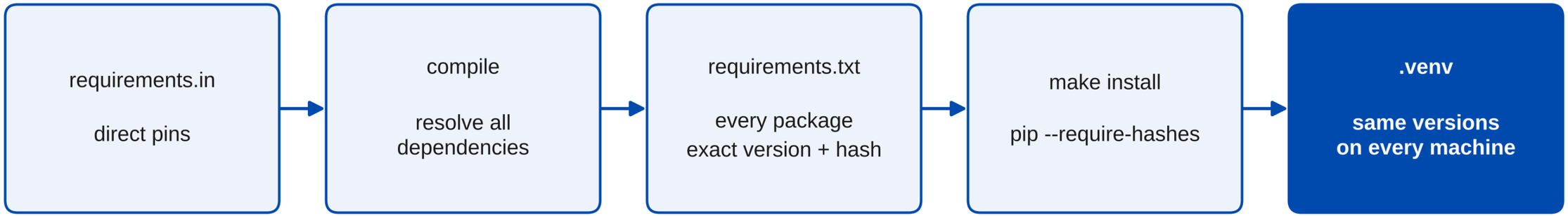
```
def load_params() -> dict:
    """Load params.yaml (seed, test_size, model hyperparameters)."""
    path = PROJECT_ROOT / "params.yaml"
    with path.open() as f:
        return yaml.safe_load(f)
```

src/diamond/config.py (lines 32-36) at tag m02

No file means `FileNotFoundError`. A test guards it.

INNOVATICS
www.innovatics.ai
Learn Data Science & AI From Zero to Hero

Pins and the lock file



pyproject.toml pins the direct packages. The lock pins everything.

Make: one name per job

```
train:
	$(PYTHON) -m diamond.train
	$(PYTHON) -m diamond.evaluate

evaluate:
	$(PYTHON) -m diamond.evaluate

test:
	$(PYTHON) -m pytest -q

lint:
	$(PYTHON) -m ruff check src tests
	$(PYTHON) -m black --check src tests

format:
	$(PYTHON) -m ruff check --fix src tests
	$(PYTHON) -m black src tests
```

Makefile (lines 16-32) at tag m02

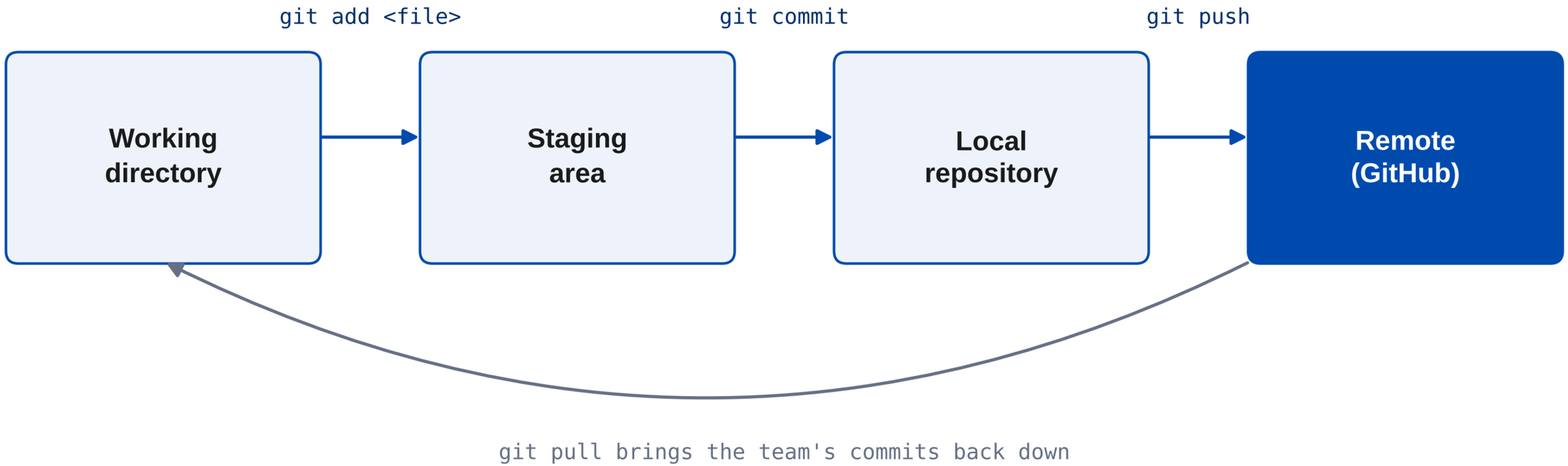
03

Git and pull requests

www.innovatics.ai
Learn Data Science & AI - From Zero to Hero

InnovatiCS 2024. All rights reserved

Three local areas and a remote



What never goes in Git

```
.venv/
.env
.env.*
__pycache__/
.ipynb_checkpoints/
notebooks/executed/
models/
*.egg-info/
build/
dist/
.pytest_cache/
.ruff_cache/
```

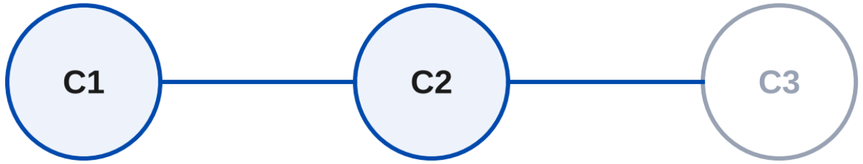
.gitignore at tag m02

www.innovatics.ai
Learn Data Science & AI From Zero to Hero

Model files are outputs. Anyone rebuilds them with `make train`.

Undo safely: reset vs revert

```
git reset --hard C2
```



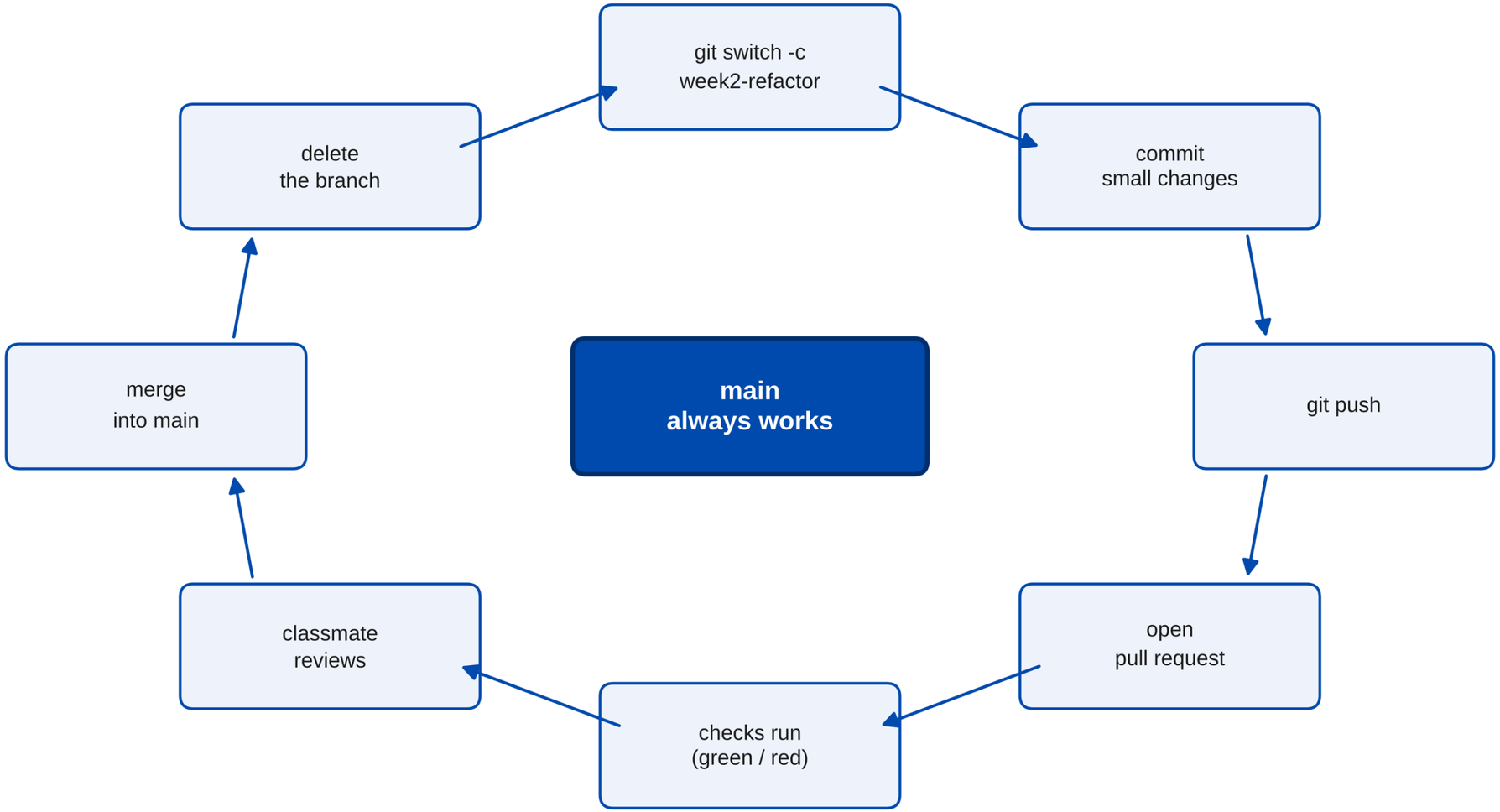
C3 is dropped. Uncommitted work is lost.
Never on pushed commits.

```
git revert --no-edit C3
```



C4 undoes C3.
History stays.
Safe after push.

GitHub Flow: the course default



What a reviewer checks

1. Does the diff do what the PR says? Read the code, not the description
2. Do tests cover the change, and is the check green?
3. Does config agree with code? No new hardcoded numbers
4. No secrets, no `.env`, no model or data files
5. Can I understand it? Clear names, small functions

INNOVATICS
www.innovatics.ai
Learn Data Science & AI From Zero to Hero

A merge conflict, by hand

params.yaml during the merge

```
<<<<<<< HEAD
    learning_rate: 0.1
=======
    learning_rate: 0.03
>>>>>>> lr-slow
```

your current branch

the branch you merge in

```
Fix: one line, no markers
    learning_rate: 0.05
```

Then: git add params.yaml, git commit, make train (MAE 279.29)

04

Code quality

www.innovatics.ai
Learn Data Science & AI - From Zero to Hero

InnovatiCS 2024. All rights reserved

Linting vs formatting

Linter: ruff

- Finds likely bugs and bad patterns
- Unused import, undefined name
- `make lint` checks, `make format` fixes
- Rules: E, W, F, I, B, UP

Formatter: black

- Rewrites layout to one style
- Never changes behavior
- Ends style arguments in reviews
- Diffs show only real changes

INNOVATICS
www.innovatics.ai
Learn Data Science & AI From Zero to Hero

Tool settings live in pyproject.toml

```
[tool.black]
line-length = 100
target-version = ["py312"]

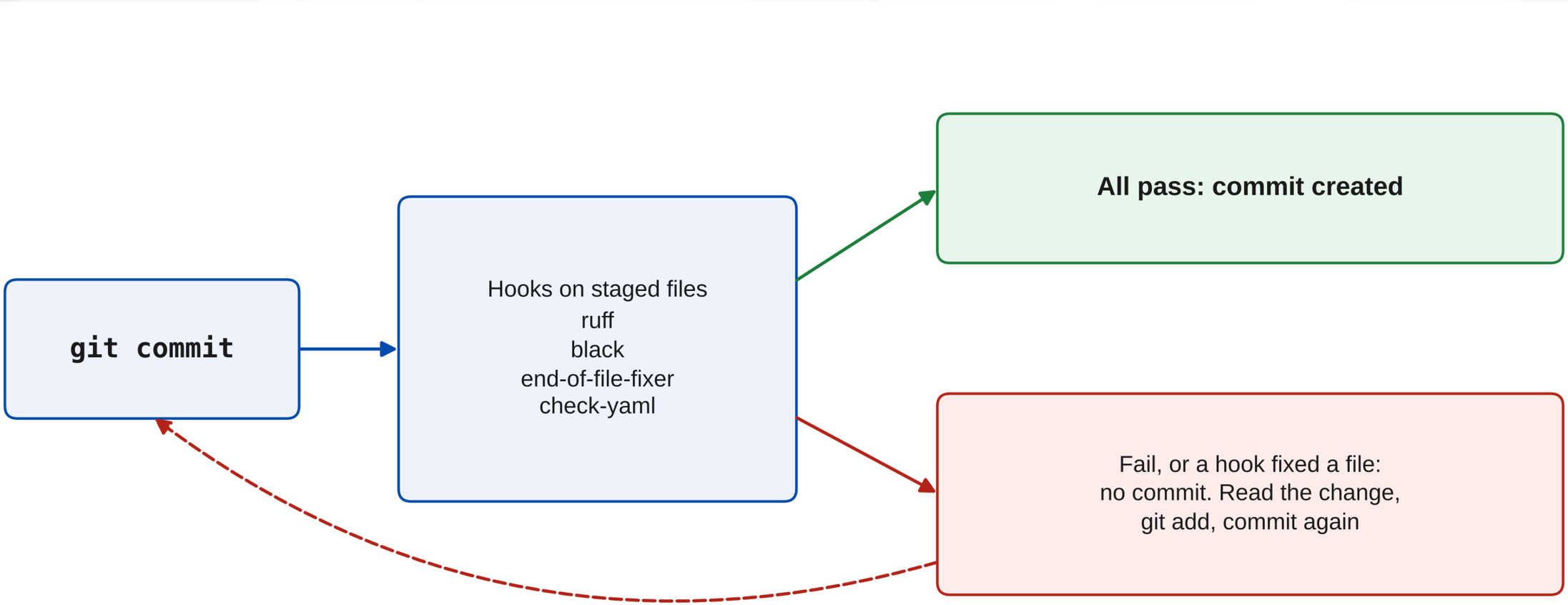
[tool.ruff]
line-length = 100
target-version = "py312"

[tool.ruff.lint]
select = ["E", "W", "F", "I", "B", "UP"]
```

pyproject.toml (lines 31-40) at tag m02

Same line length for both tools, and the rules named explicitly.

pre-commit: checks before every commit



The hooks use the locked tools

```
repos:
  - repo: local
    hooks:
      - id: ruff
        name: ruff (locked environment)
        entry: diamond-mlops/.venv/bin/ruff check --fix
        language: system
        files: ^diamond-mlops/(src|tests)/.*\.py$
      - id: black
        name: black (locked environment)
        entry: diamond-mlops/.venv/bin/black
        language: system
        files: ^diamond-mlops/(src|tests)/.*\.py$
```

.pre-commit-config.yaml (lines 3-15) at tag m02

Two more hooks follow: end-of-file-fixer and check-yaml.

05

Tests and the check

INNOVATICS
www.innovatics.ai
Learn Data Science & AI - From Zero to Hero

InnovatiCS 2024. All rights reserved

A unit test on a tiny DataFrame

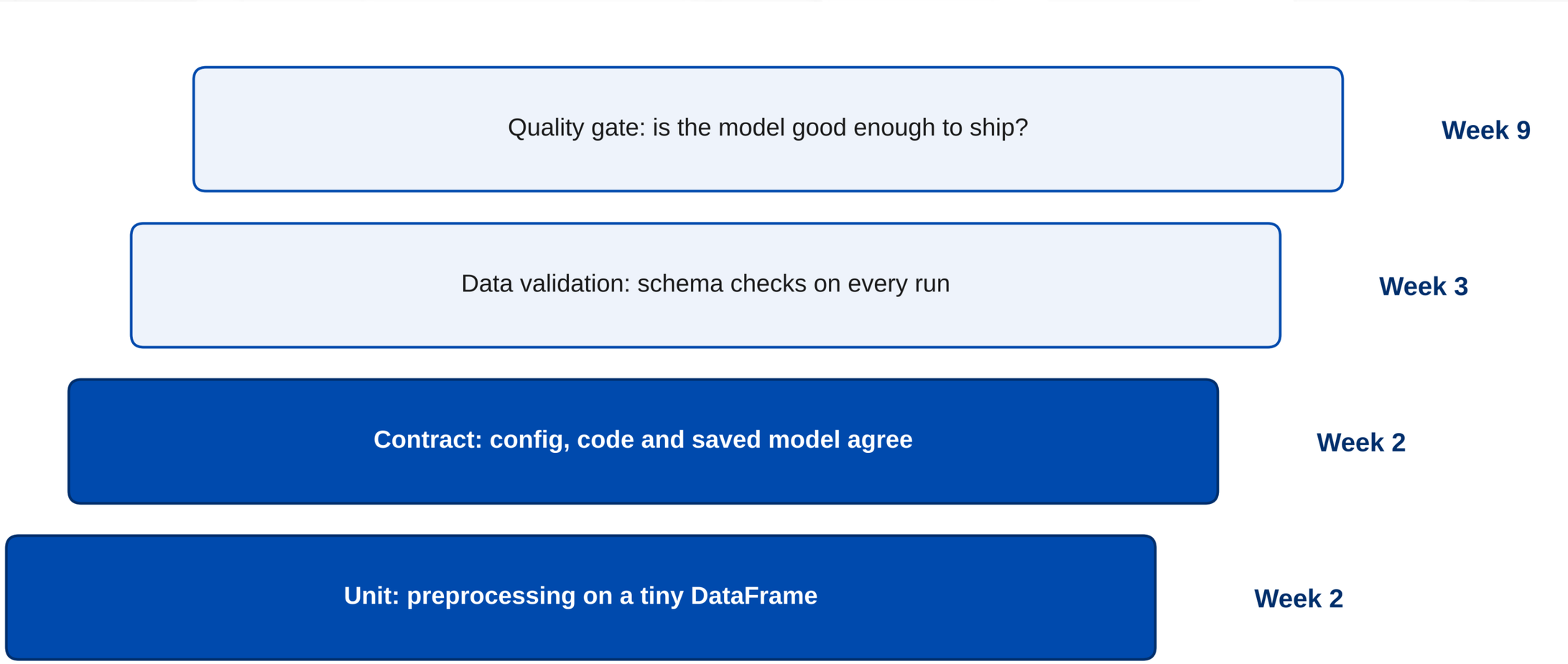
```
def test_clean_removes_duplicates_without_changing_input():
    original = _tiny_df()
    df = pd.concat([original, original.iloc[:1]], ignore_index=True)
    saved = df.copy(deep=True)
    assert len(clean(df)) == 2
    pd.testing.assert_frame_equal(df, saved)
```

tests/test_preprocess.py (lines 44-49) at tag m02

3 rows + 1 duplicate, minus 1 zero-dimension row = 2. You know the answer exactly.

INNOVATICS
www.innovatics.ai
Learn Data Science & AI From Zero to Hero

What to test this week



Proof: train twice, compare

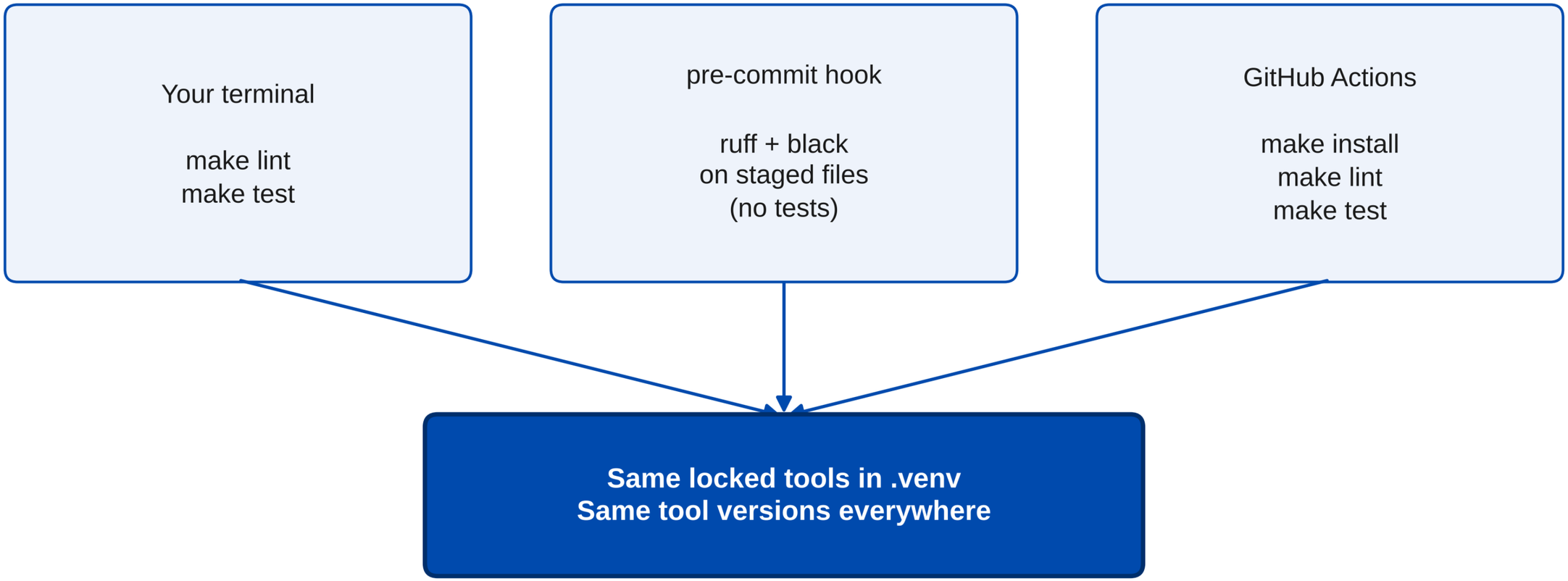
```
make train
cp models/metrics.json /tmp/diamond-metrics-first.json
make train
cmp /tmp/diamond-metrics-first.json models/metrics.json
```

README.md (lines 43-46) at tag m02

`cmp` prints nothing when the two files are identical.

INNOVATICS
www.innovatics.ai
Learn Data Science & AI From Zero to Hero

Same tools in three places



Alternatives

Job	Ours	Others, and when to pick them
Dependencies	venv + pip + hash lock	conda: non-Python libraries. poetry, uv: tool manages pins and lock
Lint and format	ruff + black	flake8 + isort + pylint: older projects. `ruff format`: one tool for both
Branching	GitHub Flow	Trunk-based: strong CI, tiny branches. Git Flow: scheduled releases
Config	plain YAML	Hydra: many config groups and command-line overrides

Common mistakes

You see	Cause	Fix
`missing separator.  Stop.`	Spaces in a Makefile recipe	Use a tab
Commit "vanished"	A hook fixed a file	`git add` it, commit again
`NotImplementedError: TODO M02`	A TODO is still open	Finish the function, rerun
Red check, green locally	Not pushed, or workflow in the wrong folder	Push; keep it at the repo root
`cmp` prints a difference	`params.yaml` or packages changed	`git diff`, `make install`
YAML error after a merge	Conflict marker left in the file	Delete every marker line

Practical 2: notebook to tested project

- Start from the Week 2 starter branch. Work on `week2-refactor`
- Fill 3 TODOs: fit the model, score the holdout, finish one test
- Prove it: `make train` twice and `cmp` prints nothing; `make test` gives 13 passed
- Do the merge-conflict exercise. The known answer brings MAE back to 279.29
- Open a PR, get a green check and one classmate review, then merge

INNOVATICS
www.innovatics.ai
Learn Data Science & AI From Zero to Hero

Quiz 2 and next week

- Quiz 2 opens now: reproducibility, layout, Git, lint vs format, pre-commit, testing
- Next week: data management. Validate the data, version it with DVC
- `dvc repro` reruns only what changed, and reads `params.yaml`
- Bring a finished Practical 2. Week 3 starts from it
- Required reading: GitHub Docs, "Resolving a merge conflict using the command line"

INNOVATICS
www.innovatics.ai
Learn Data Science & AI From Zero to Hero

THANK YOU



InnovatiCS 2024. All rights reserved
+1 (678) 209-9780